

PART II — BUILDING THE INGESTION PIPELINE

Chapter 8

Embeddings – Deep Dive

“To embed is to compress; to retrieve is to unfold.”

Chapter 8 — Embeddings – Deep Dive

Chapter 7 left you with a pile of clean text chunks — short, bounded strings that have been stripped of boilerplate and verified for minimum length. The next step is the conceptual heart of every RAG system: converting those strings into numbers. Not arbitrary numbers, but numbers that encode meaning. This chapter is entirely about that conversion, what it produces, where the machinery comes from, and how to use it without triggering memory failures, wasting API budget, or choosing the wrong model for your corpus.

By the end of this chapter, you will understand what a dense vector actually is and why it works. You will be able to trace the full journey from raw sentence to normalised 384-dimensional float array. You will know how to choose between MiniLM, BGE, OpenAI, Cohere, and Voyage. And you will have a production-ready `EmbeddingManager` class that handles batching, caching, and cosine similarity in one place.

The sections run in dependency order. Sections 8.1 through 8.3 build the conceptual foundation. Sections 8.4 and 8.5 survey the embedding model landscape and the tradeoffs among them. Sections 8.6 and 8.7 tackle operational concerns: speed and memory. Section 8.8 shows you how to evaluate a model on your own data before committing to it. Sections 8.9 and 8.10 implement the `EmbeddingManager` class and explain the mathematics that makes L2 normalisation non-optional in this stack.

8.1 Dense Vectors as Compressed Meaning

Language is continuous and contextual. The word “bank” means something different when preceded by “river” than when preceded by “investment.” A keyword index treats both occurrences identically. A dense vector does not. The vector for “river bank” will sit in a different region of the embedding space than the vector for “investment bank”, because the model that produced it has learned, from hundreds of millions of training sentences, that these contexts cluster differently.

A dense vector is a fixed-length array of floating-point numbers — typically 384, 768, or 1536 values — where every coordinate participates in encoding the meaning of the input. The word “dense” distinguishes it from a sparse vector (such as a TF-IDF or BM25 representation) in which most coordinates are zero. In a dense vector there are no zero coordinates by design; every dimension carries a small piece of the semantic signal.

Definition — Dense Vector (Embedding)

A dense vector, or embedding, is a fixed-length array of real numbers produced by a neural network encoder in response to an input string. Each coordinate encodes a fragment of the semantic content of that string. Vectors for semantically similar inputs cluster geometrically: their cosine similarity is high. Vectors for unrelated inputs are nearly orthogonal: their cosine similarity approaches zero.

Formally: $e: \mathbb{R}^n \rightarrow \mathbb{R}^d$, where n is the variable length of the token sequence and d is the fixed embedding dimension (e.g., 384).

The key property is that geometric proximity corresponds to semantic proximity. You can ask “which stored vectors are nearest to this query vector?” and obtain chunks whose meaning is close to the query's meaning, even if no word in common exists. That is the retrieval primitive on which all of RAG is built.

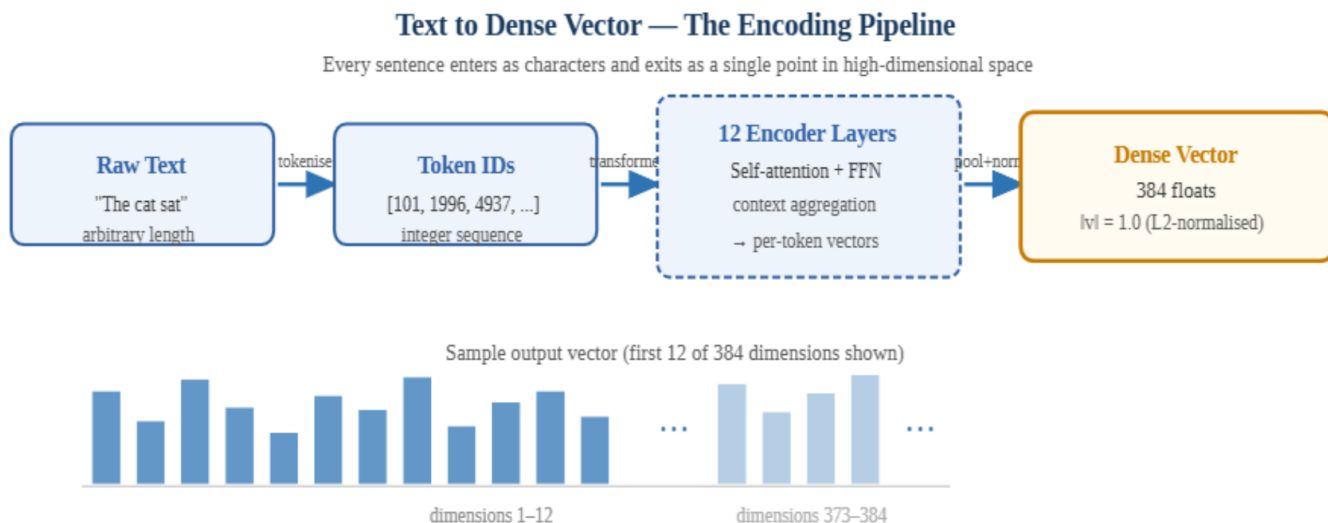


Figure 8.1 — The text-to-vector pipeline. Raw text enters as a character sequence of arbitrary length; a fixed-size, normalised float array exits. Every downstream component — the vector store, the retriever, the ranker — sees only that array.

Analogy — The museum floor plan

Imagine a vast museum where every exhibit is a sentence. Semantically similar sentences are hung in the same wing. If you walk into the “machine learning” gallery, every item on the walls discusses training, gradients, and parameters. The room for “cooking techniques” is on a different floor entirely. A dense vector is the GPS coordinate of an exhibit in that museum — a compact address that tells you exactly where it hangs, without any of the exhibit's actual content. Retrieval is just finding the exhibits closest to where your query hangs.

8.2 How sentence-transformers Produce a 384-d Vector

The sentence-transformers library wraps a family of transformer models that have been fine-tuned using a specific objective: make vectors for semantically equivalent sentences as similar as possible, and vectors for semantically unrelated sentences as dissimilar as possible. The canonical development model in this project is all-MiniLM-L6-v2, a distilled BERT-like encoder trained on over one billion sentence pairs.

The internal process unfolds in four stages. In the first stage, the input string is tokenised by a WordPiece tokeniser that splits the text into subword units and prepends a special [CLS] token. In the second stage, each token ID is looked up in a 30,522-entry embedding matrix, adding positional embeddings so the model retains word-order information. In the third stage, the sequence passes through six encoder layers. Each layer applies multi-head self-attention — six heads, each attending over the entire token sequence — followed by a position-wise feed-forward network and layer normalisation. The result is one 384-dimensional vector per input token. In the fourth stage, a mean-pooling operation averages all token vectors into a single 384-d sentence vector, which is then L2-normalised so its magnitude equals exactly 1.0.

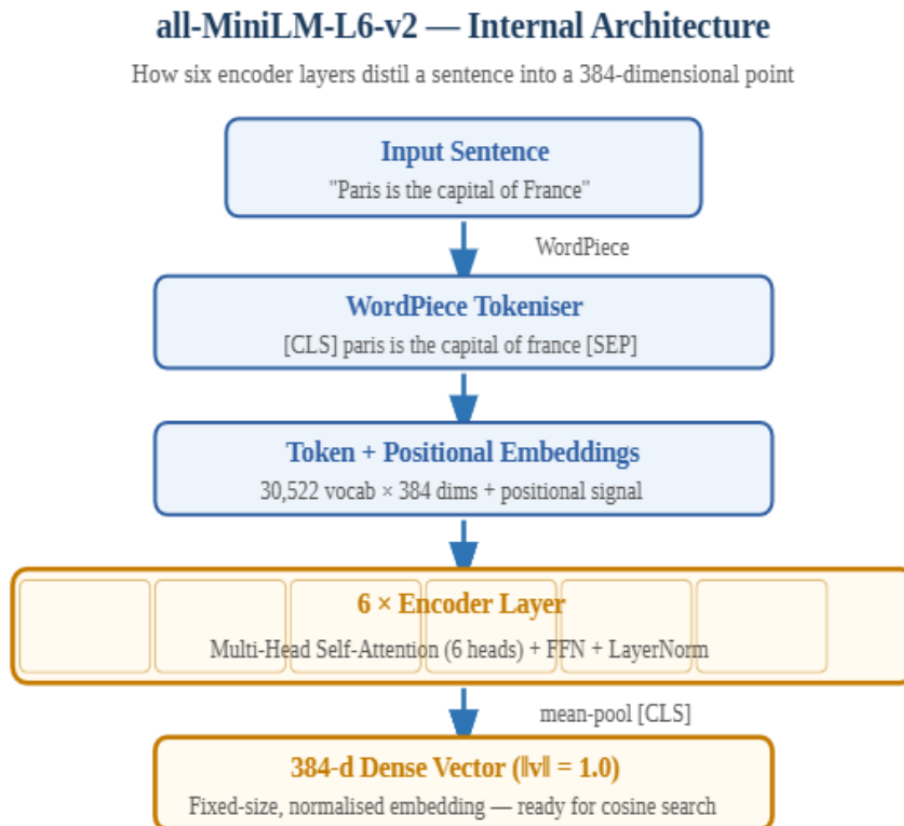


Figure 8.2 — The all-MiniLM-L6-v2 architecture. Six encoder layers process a tokenised sentence and output one 384-d vector per token; mean-pooling collapses those into the sentence embedding.

Definition — Mean Pooling

Mean pooling is the operation that converts a sequence of per-token vectors into a single sentence vector. Given an input of T tokens, each represented by a vector $v^i \in \mathbb{R}^d$, mean pooling computes the element-wise arithmetic mean: $\text{sentence_vec} = (1/T) \times \sum v^i$. Attention-mask-aware implementations weight each token's contribution by its mask value (0 for padding tokens, 1 for real tokens) so that padding does not dilute the result.

The model encodes context holistically. Unlike early word2vec models that produced one vector per word regardless of context, a sentence-transformer reads the entire input in one forward pass. The vector for “bank” in the sentence “the river bank flooded” is numerically different from the vector for “bank” in “the bank rate rose,” because the surrounding context shifts the attention patterns in every layer.

```

from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
vec = model.encode('Paris is the capital of France',
                  normalize_embeddings=True)
# vec.shape -> (384,)
# np.linalg.norm(vec) -> 1.0000001 (floating-point rounding)
  
```

8.3 Where the Model Comes From — Hugging Face Hub and Local Caching

When you instantiate `SentenceTransformer('all-MiniLM-L6-v2')` for the first time on a machine, the library contacts the Hugging Face Hub, downloads the model's weights and configuration files over HTTPS, and stores them on disk in a structured cache directory. Every subsequent instantiation on the same machine reads from that cache instantly — no network call, no download latency.

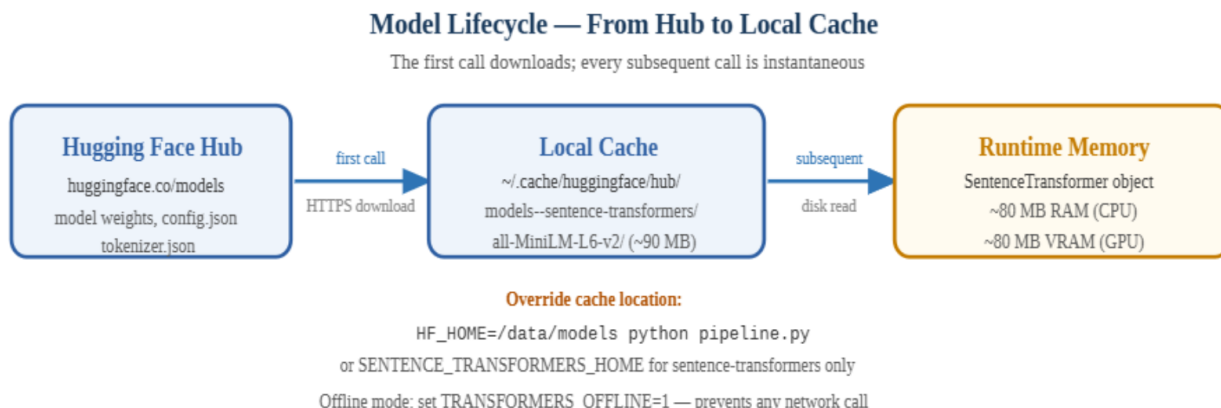


Figure 8.3 — Model lifecycle from Hugging Face Hub to runtime memory. The first call incurs a download; all subsequent calls are local disk reads from the cache at `~/.cache/huggingface/hub/`.

The default cache root is `~/.cache/huggingface/hub/` on Linux and macOS, and `%USERPROFILE%\cache\huggingface\hub\` on Windows. The `all-MiniLM-L6-v2` model weighs approximately 90 MB on disk. If you need to relocate the cache — because the home directory is on a small SSD while your data drive is larger — set the environment variable `HF_HOME` to any path before importing the library.

```
import os
os.environ['HF_HOME'] = '/data/models' # set before import
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2') # writes to /data/models/

# Prevent any network call (airgapped environment):
os.environ['TRANSFORMERS_OFFLINE'] = '1'
os.environ['HF_DATASETS_OFFLINE'] = '1'
```

Common pitfall — Cache fills the root partition

On many cloud instances, the home directory lives on the root partition (10–20 GB). Once you embed a few large models, the cache can silently consume all available space, causing Python to crash mid-inference with a disk-full `OSError` rather than a friendly message. Always point `HF_HOME` at a data volume if your home partition is smaller than 50 GB. Run `du -sh ~/.cache/huggingface` periodically to monitor growth.

8.4 Open-Source vs Paid Embedding Models

Embedding models span a wide range of capabilities, costs, and operational requirements. The decision between open-source and hosted-API models is not purely technical — it involves budget, data privacy, deployment complexity, and the quality demands of your retrieval workload.

Open-Source Models

Open-source models run locally or on your own infrastructure. You pay only for compute, and your documents never leave your environment, which matters for sensitive data. The main families relevant to this book are:

- all-MiniLM-L6-v2 (384 dims, ~90 MB). The default development model in this project. Fast, CPU-friendly, and accurate enough for most general-domain corpora. Best choice for prototyping and resource-constrained deployments.
- all-MiniLM-L12-v2 (384 dims, ~130 MB). Twelve encoder layers instead of six. Measurably better retrieval quality at the cost of approximately 1.6× inference time. Worth the upgrade once your pipeline is stable.
- BGE-M3 (1024 dims, ~570 MB). From the Beijing Academy of AI. State-of-the-art multilingual performance and strong English benchmarks. Supports three retrieval modes (dense, sparse, and multi-vector) in a single model. Recommended when quality is the primary constraint and GPU is available.
- Nomic-embed-text-v1 (768 dims, ~280 MB). Fully open-source with a permissive Apache 2.0 license and competitive MTEB scores. Supports 8192-token context window, making it useful for long documents.

Paid / Hosted Models

Hosted embedding APIs offload the inference burden to the provider's servers. You send text, receive vectors, and pay per token. The main providers in 2025 are:

- OpenAI text-embedding-3-small (1536 dims) and text-embedding-3-large (3072 dims). The large variant consistently ranks at the top of the MTEB leaderboard. Matryoshka representation learning allows truncating the vector to lower dimensions without retraining. Price: ~\$0.02 per million tokens (small), ~\$0.13 per million (large).
 - Cohere embed-english-v3.0 (1024 dims). Natively supports an `input_type` parameter that separately optimises embeddings for indexing (`search_document`) versus querying (`search_query`), improving retrieval precision. Strong performance on domain-specific corpora.
 - Voyage AI voyage-large-2 (1536 dims). Specialised for retrieval tasks. Voyage-code-2 is purpose-built for code corpora. Price: ~\$0.12 per million tokens.
-

EmbeddingManager — Class Structure and Data Flow

The wrapper that turns a model name into query-ready vectors

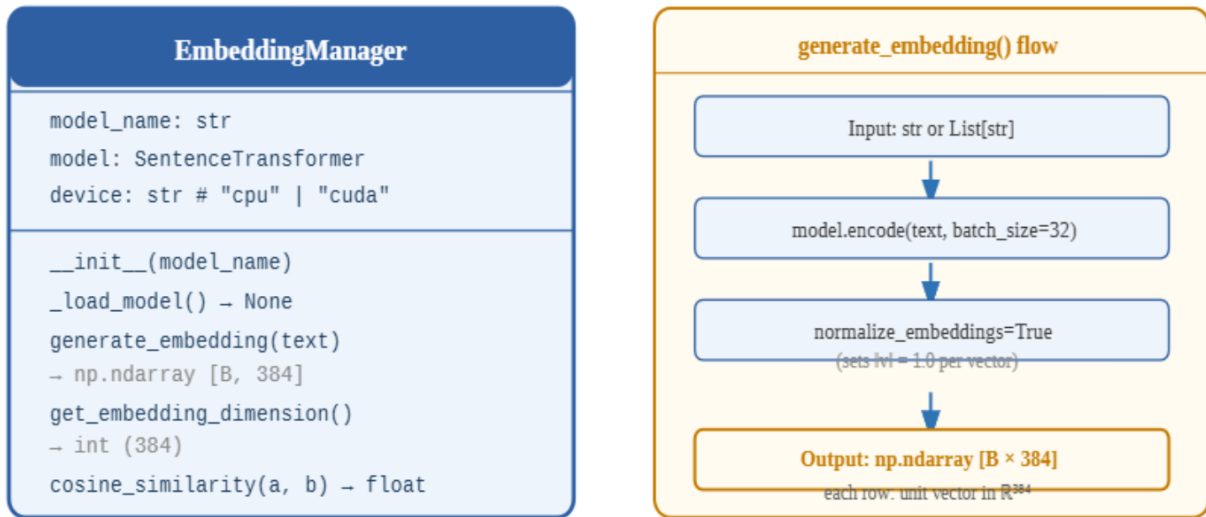


Figure 8.4 — The *EmbeddingManager* class structure and the data flow through `generate_embedding()`. The wrapper normalises every output automatically, so callers never receive non-unit vectors.

8.5 Cost, Dimensionality, and Quality Tradeoffs

Choosing an embedding model requires balancing three variables that pull in different directions: embedding quality (how well similar texts cluster), dimensionality (how much memory and storage the vectors consume), and cost (compute time for local models or API price for hosted ones). The table below organises the main options across these axes.

Model	Dims	MTEB Score	Cost Type	Best For
all-MiniLM-L6-v2	384	56.3	Local/Free	Dev & low-resource prod
all-MiniLM-L12-v2	384	58.8	Local/Free	Better accuracy, same dims
BGE-M3	1024	65.0+	Local/Free	Multilingual, GPU available
Nomic-embed-text-v1	768	62.4	Local/Free	Long docs (8K tokens)
OpenAI embed-3-small	1536	62.3	API ~\$0.02/M tok	Balance quality & cost
OpenAI embed-3-large	3072	64.6	API ~\$0.13/M tok	Top quality, small corpus
Cohere embed-v3.0	1024	64.5	API ~\$0.10/M tok	Domain-specific corpora
Voyage large-2	1536	65.1	API ~\$0.12/M tok	Highest retrieval accuracy

Dimensionality has two practical effects. Higher-dimensional vectors require more storage in the vector database (a 3072-d float32 vector consumes 12 KB versus 1.5 KB for a 384-d vector), and similarity search time grows with dimension. For a corpus of 100,000 chunks, this difference is negligible. For ten million chunks, it determines whether your index fits in RAM. Local open-source models of 384 or 768 dimensions are almost always sufficient unless you have strong evidence that your retrieval quality is bottlenecked by model expressiveness rather than chunking, prompting, or reranking.

Analogy — The resolution vs storage tradeoff

Think of embedding dimensionality like image resolution. A 384-dimension embedding is a 480p video: it captures the scene well enough for most purposes. A 3072-dimension embedding is 4K: more detail, better fidelity, four times the storage, four times the bandwidth. Before paying for 4K, verify that your screen can actually display the difference. For most RAG workloads, the retrieval bottleneck is not embedding resolution — it is chunking quality and prompt design.

8.6 Batching Embeddings for Speed Without Crashing Memory

`SentenceTransformer.encode()` accepts both a single string and a list of strings. When passed a list, it processes the strings in internal mini-batches and returns a two-dimensional array with one row per input. Always pass lists rather than strings in a loop: the per-call overhead (Python function call, input validation, GPU kernel launch) is paid once per `encode()` call regardless of how many strings are inside.

```
# SLOW: N separate encode() calls
vectors = [model.encode(text) for text in texts]

# FAST: one encode() call with a list
vectors = model.encode(texts,          # List[str]
                        batch_size=32, # texts per GPU mini-batch
                        show_progress_bar=True,
                        normalize_embeddings=True,
                        convert_to_numpy=True)

# vectors.shape -> (N, 384)
```

The `batch_size` parameter controls how many strings the GPU (or CPU vectorised kernel) processes simultaneously. Larger batches reduce the number of kernel launches but increase peak memory consumption. The practical ceiling on a machine with 8 GB RAM and no GPU is typically `batch_size=32` to 64. With a 4 GB VRAM GPU, you can safely use `batch_size=128` for short texts; for long texts (500+ tokens), reduce to 32 to avoid out-of-memory errors.

Common pitfall — Embedding the entire corpus at once

Calling `model.encode(all_chunks)` where `all_chunks` contains 50,000 items and trusting the internal batching to manage memory is tempting but risky. The library batches the forward passes, but it allocates the entire output array — $50,000 \times 384$ float32 values — in memory upfront. That is 74 MB, acceptable, but if the texts are long (averaging 300 tokens), the tokenised input tensor dwarfs the output. The safe pattern is to iterate in explicit outer batches of 256–1000 items, embed each batch, and append the results to a list or write them directly to the vector store.


```
import numpy as np

OUTER_BATCH = 512    # chunks per outer iteration

def embed_in_batches(texts, model, inner_batch=32):
    all_vecs = []
    for i in range(0, len(texts), OUTER_BATCH):
        chunk = texts[i : i + OUTER_BATCH]
        vecs = model.encode(chunk,
                             batch_size=inner_batch,
                             normalize_embeddings=True,
                             convert_to_numpy=True)
        all_vecs.append(vecs)
        print(f'Embedded {min(i+OUTER_BATCH, len(texts))}/{len(texts)}')
    return np.vstack(all_vecs)
```

8.7 Caching and Reusing Embeddings

Embedding is deterministic: the same model weights applied to the same input always produce the same output. This means that re-embedding a corpus you have already ingested is pure waste. Two caching strategies avoid that waste at different granularities.

Strategy 1 — Persist to the vector store and skip re-ingestion

The most effective cache is the vector store itself. Once a chunk has been embedded and inserted, its vector is persisted. Incremental ingestion logic (see Chapter 10) detects whether a document has changed since the last run by comparing file modification timestamps or content hashes. Only new or modified documents are re-embedded. This strategy requires zero additional infrastructure.

Strategy 2 — An in-memory or disk-based embedding cache

For interactive workloads where the same query or the same chunk may be embedded multiple times across multiple sessions, a lightweight disk cache using Python's `shelve` module or the `diskcache` library eliminates repeated model calls. The cache key is a hash of the model name concatenated with the input text; the value is the numpy array, serialised as bytes.

```
import hashlib, shelve, numpy as np

CACHE_PATH = '../data/embed_cache'

def cached_embed(text: str, model, cache_path=CACHE_PATH):
    key = hashlib.sha256(
        (model.model_card_data.model_name + text).encode()
    ).hexdigest()
    with shelve.open(cache_path) as db:
        if key in db:
            return np.frombuffer(db[key], dtype=np.float32)
        vec = model.encode(text, normalize_embeddings=True,
                           convert_to_numpy=True)
        db[key] = vec.tobytes()
    return vec
```

Common pitfall — Caching across model versions

A cache key that does not include the model name or model revision will silently serve vectors from a previous, different model when you upgrade. Always include both the full model identifier and the specific revision (commit hash) in the cache key. HuggingFace models expose the revision via `model.model_card_data` or through the `model_info()` API. Mixing vectors from two different model versions in the same ChromaDB collection corrupts similarity scores in ways that are extremely difficult to debug.

8.8 Evaluating an Embedding Model on Your Own Data

MTEB (Massive Text Embedding Benchmark) scores give you a reasonable prior, but they are measured on fixed public datasets. Your domain may differ significantly — a legal contract corpus, a proprietary technical manual library, or a customer support ticket archive all have vocabulary distributions that diverge from MTEB's benchmarks. Before committing to a model for production, evaluate it on a small labelled sample from your own corpus.

The minimum viable evaluation requires three things: a set of queries representative of what your users will actually ask, a set of relevant and non-relevant chunks for each query (ground-truth labels), and a scoring function. The most informative metrics for RAG are Hit Rate@k (does at least one relevant chunk appear in the top-k results?) and Mean Reciprocal Rank (how high does the first relevant result appear on average?).

```
# Minimal embedding model evaluation on your corpus
from sentence_transformers import SentenceTransformer
import numpy as np

def hit_rate_at_k(model, queries, relevant_ids, corpus_texts,
                  corpus_ids, k=5):
    corpus_vecs = model.encode(corpus_texts,
                               normalize_embeddings=True)

    hits = 0
    for query, rel_ids in zip(queries, relevant_ids):
        q_vec = model.encode(query, normalize_embeddings=True)
        scores = corpus_vecs @ q_vec  # dot product = cosine (normalised)
        top_k = np.argsort(scores)[::-1][:k]
        top_ids = [corpus_ids[i] for i in top_k]
        if any(rid in top_ids for rid in rel_ids):
            hits += 1
    return hits / len(queries)
```

Run this function on a labelled set of 50–200 query–relevant pairs. If your domain is specialised enough that out-of-the-box MiniLM scores below 0.60 Hit Rate@5, consider BGE-M3 or a paid hosted model. If those too fall short, domain-adaptive fine-tuning via sentence-transformers' training API is the next step — beyond the scope of this chapter but covered in Chapter 21.

8.9 Building the EmbeddingManager Class

The EmbeddingManager class is the single component in this codebase responsible for all embedding operations: loading the model, generating vectors, reporting the embedding dimension, and computing cosine similarity. Every other class — VectorStore, RAGRetriever, SelfLearner — receives an EmbeddingManager instance through constructor injection and calls only its public methods. This strict containment means you can swap the underlying model by changing one argument without touching any other file.

EmbeddingManager — Class Structure and Data Flow

The wrapper that turns a model name into query-ready vectors

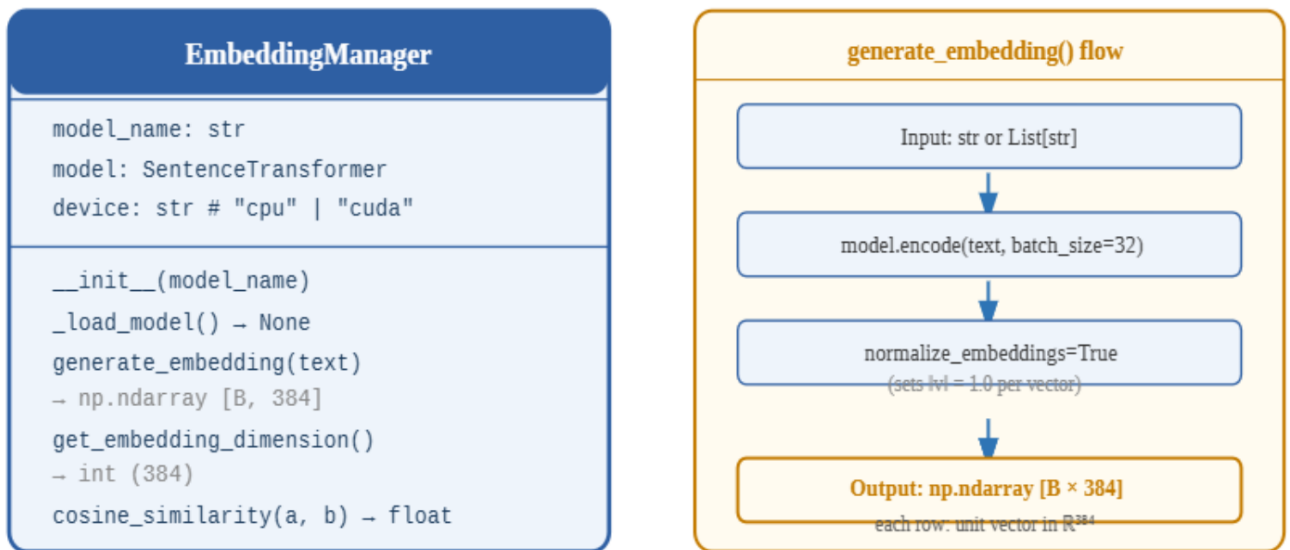


Figure 8.5 — EmbeddingManager class diagram. The class encapsulates model loading, device selection, embedding generation, and cosine similarity. All normalisation happens inside generate_embedding() so callers never receive non-unit vectors.

The implementation below is the production version from `embedding_manager.py` in this project. Read it section by section before moving to Section 8.10, which explains the mathematical reason that `normalize_embeddings=True` is unconditional.

```
import numpy as np
import torch
from sentence_transformers import SentenceTransformer

class EmbeddingManager:
    def __init__(self, model_name: str = 'all-MiniLM-L6-v2'):
        self.model_name = model_name
        self.model = None
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
        self._load_model()
    def _load_model(self):
        try:
            self.model = SentenceTransformer(
                self.model_name, device=self.device)
            dim = self.model.get_sentence_embedding_dimension()
            print(f"Model '{self.model_name}' loaded on {self.device}.")
            print(f"Embedding dimension: {dim}")
        except Exception as e:
            print(f"Error loading model: {e}")
            raise
    def generate_embedding(self, text,
                          batch_size: int = 32,
                          show_progress: bool = False
                          ) -> np.ndarray:
        if self.model is None:
            raise ValueError('Model not loaded.')
        return self.model.encode(
            text,
            batch_size=batch_size,
            show_progress_bar=show_progress,
            convert_to_numpy=True,
            normalize_embeddings=True,    # ALWAYS True
            device=self.device,
        )
    def get_embedding_dimension(self) -> int:
        if self.model is None:
            raise ValueError('Model not loaded.')
        return self.model.get_sentence_embedding_dimension()

    def cosine_similarity(self,
                          vec_a: np.ndarray,
                          vec_b: np.ndarray) -> float:
        if vec_a.shape != vec_b.shape:
            raise ValueError('Vectors must have the same shape.')
        norm_a = np.linalg.norm(vec_a)
        norm_b = np.linalg.norm(vec_b)
        if norm_a == 0 or norm_b == 0:
            return 0.0
        return float(np.dot(vec_a, vec_b) / (norm_a * norm_b))
```

Three design choices in this implementation deserve explicit comment. First, device detection runs at construction time; callers do not need to know whether a GPU is available. Second, `normalize_embeddings=True` is unconditional inside `generate_embedding()` — it cannot be overridden by callers. This is deliberate: a non-normalised vector entering the vector store would corrupt cosine similarity scores globally. Third, `cosine_similarity()` checks for zero-norm vectors before dividing, preventing a divide-by-zero error that would otherwise surface as a runtime exception in the middle of a retrieval pipeline.

8.10 Why Vectors are L2-Normalised and What $\|v\| = 1.0$ Actually Means

L2 normalisation divides a vector by its Euclidean (L2) norm, shrinking or stretching it so that its length becomes exactly 1.0. The Euclidean norm of a vector $v = [v_1, v_2, \dots, v_d]$ is the square root of the sum of squared coordinates: $\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_d^2}$. After normalisation, the vector $\hat{v} = v / \|v\|$ satisfies $\|\hat{v}\| = 1.0$. Geometrically, all normalised vectors lie on the surface of a d-dimensional unit hypersphere centred at the origin.

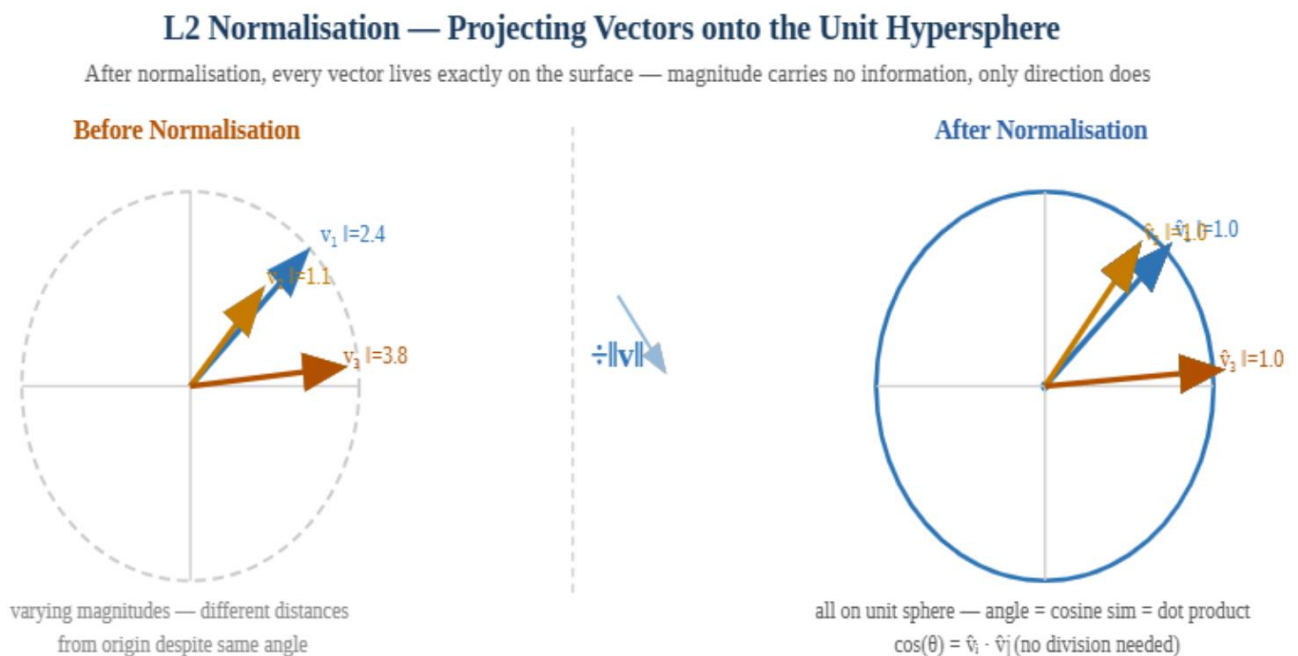


Figure 8.6 — L2 normalisation in 2D cross-section. Before normalisation, vectors have different magnitudes and sit at different distances from the origin. After normalisation, every vector lands on the unit circle. Only direction — angle — survives.

Definition — L2 Norm and Normalisation

The L2 norm (Euclidean norm) of a vector $v \in \mathbb{R}^d$ is $\|v\|_2 = \sqrt{\sum_i v_i^2}$. L2 normalisation produces the unit vector $\hat{v} = v / \|v\|_2$. The resulting vector satisfies $\|\hat{v}\|_2 = 1.0$ and encodes only direction, not magnitude. Two unit vectors $\hat{v}$ and $\hat{w}$ satisfy: $\cos(\theta) = \hat{v} \cdot \hat{w} = \sum_i \hat{v}_i \hat{w}_i$. Because both norms equal 1, the cosine similarity reduces to a pure dot product — the fastest possible similarity computation on modern hardware.

The practical payoff of normalisation is computational. The full cosine similarity formula is $\cos(\theta) = (a \cdot b) / (\|a\| \times \|b\|)$. When both vectors are unit vectors, both norms equal 1.0, so the formula simplifies to $\cos(\theta) = a \cdot b$. A dot product is a single matrix multiplication, which GPUs and BLAS libraries execute in highly optimised vectorised code. The division, which would require computing two separate norms and performing element-wise scaling, disappears entirely. At the scale of searching a million stored vectors in milliseconds, that optimisation is the difference between a responsive system and one that times out.

There is also a semantic reason for normalisation. Without it, long texts would systematically produce vectors with larger norms than short texts simply because they accumulate more activation mass through the mean-pooling step. A three-paragraph document would appear “closer” to every query than a two-sentence summary of the same content, purely because of length, not meaning. Normalisation removes that artefact: every vector lives at unit distance from the origin, and similarity is determined entirely by direction — by meaning — not by input length.

Common pitfall — Mixing normalised and non-normalised vectors

If you insert some vectors into ChromaDB with `normalize_embeddings=True` and others without, the collection silently holds vectors of different magnitudes. Cosine similarity queries will still run without errors — ChromaDB applies the formula correctly — but results will be skewed toward the non-normalised vectors, whose larger norms inflate their dot products with every query. The symptom is retrieval results that seem biased toward a subset of your documents with no obvious semantic reason. The fix is to rebuild the collection entirely from a fresh embed run with normalisation enforced.

A final note on floating-point precision: normalizing a vector in float32 does not produce $\|\hat{v}\| = 1.0$ exactly. It produces a value such as 0.9999999 or 1.0000001. This is expected and harmless. The deviations are on the order of 10^{-7} and have no measurable effect on cosine similarity scores. If you write an assertion $\|\hat{v}\| == 1.0$, it will sometimes fail. Write it as $\text{abs}(\|\hat{v}\| - 1.0) < 1\text{e-}5$ instead.
